

Document number: P3172R0
Date: 2024-03-08
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Using `this` in constructor preconditions

[\[P2900R6\]](#) (Contracts for C++), while it doesn't state it explicitly, allows the evaluation of member functions in constructor preconditions. Constructor preconditions are evaluated before the execution of the constructor starts, before calling constructors of member and base subobjects, so evaluating member functions at that point is likely to break unless used with caution. This paper proposes two ways to address this: either explicitly call this undefined behavior or make it ill-formed.

1. Problem description

Constructor preconditions would typically express the constraints on function parameters, and reflect the constraints of the subobject's constructors.

```
class X
{
    std::string name;

public:
    explicit X(const char * n)
        pre(name != nullptr) // evaluated first
        : name{n}           // evaluated second
    {}
};
```

[\[P2900R6\]](#) also in general allows the (potentially implicit) usage of `this` in class members' preconditions, to cover common cases like this:

```
T& container<T>::front()
    pre(!empty());
```

But combining the two properties cannot work. Analogous observation applies to postconditions of destructors.

Two features in C++ already suffer from a similar problem. The first is calling — possibly indirectly — virtual functions in constructors and destructors. In those cases the virtual function call mechanism will not be called, skipping the overriding functions, and in the worst case we will get a *pure virtual function call*.

```
struct X
{
    virtual void f() = 0;
    void g() { f(); }
    X() { g(); }
};

struct Y : X
{
```

```
void f() override {}
};
```

The second feature observing a similar problem is function-try-blocks. It allows you to execute a code in the constructor body, even though member and base subobjects may not have been initialized.

```
struct B
{
    unique_ptr<int> p;
    // invariant: p != nullptr

    B() {
        throw "error"; // failure during initialization
        p = make_unique<int>(1);
    }
};

struct D : B
{
    D()
    try {}
    catch (...) {
        cout << *p; // initialized or not?
    }
};
```

Referring to any non-static member or base class of an object in the handler for a *function-try-block* of a constructor or destructor for that object results in undefined behavior ([\[except.handle\]/10](#)).

2. Solutions

2.1. Undefined behavior

The first possible solution is to do what C++ currently does for virtual functions and function-try-blocks: just *trust the user* that they will not do the dangerous things. In other words, make it undefined behavior when a precondition in a constructor or a postcondition in a destructor of object *X* refers to any non-static member or base class of *X*, or does anything else interacting with the construction/destruction process. This allows users that know what they are doing to express preconditions on constructors in rare cases where this actually makes sense. One such example has been provided by Gašper Ažman

```
template <class FullType>
struct MixinA
{
    int size() const;
};

template <typename FullType>
struct MixinB
{
    MixinB()
        pre(static_cast<FullType const*>(*this).size() > 0);
};
```

```
struct FT : MixinA<FT>, MixinB<FT>
{};
```

Here `MixinB` is aware of the existence of `MixinA` and that `MixinA` will have been initialized before the constructor of `MixinB` starts. We only access `*this` to access a fully constructed subobject.

It should be noted that the above scheme could be refactored so that `MixinB` receives a reference to `MixinA` in the constructor, and then the implicit reference to `*this` could be avoided. But the general point still holds.

Another motivating example, that one would expect to work fine, by Joshua Berne:

```
class SelfRegistering
{
public:
    SelfRegistering()
        pre( !Registry::isRegistered(this) )
        post( Registry::isRegistered(this) );

    ~SelfRegistry()
        pre( Registry::isRegistered(this) )
        post( !Registry::isRegistered(this));
};
```

Yet another example, by Lisa Lippincott, is for a class used for cryptography, representing a secret, to check in a constructor precondition if its storage is allocated in a dedicated secure partition, and to check in the destructor postcondition that the storage (not value) has been zeroed out.

Currently C++ has two level of "strictness" when defining the behavior of using objects during construction. A stricter level applies until all base class subobjects have been initialized. A less strict one applies after all the base class subobjects have been initialized:

```
struct D : B1, B2
{
    M m1, m2;
    auto f(X x);

public:
    D(X x)
        : B1(f(x))      // UB
        , B2(f(x))      // UB
        , m1(f(x))      // <-- less strict part starts
        , m2(f(x))      // not UB
    {}
};
```

2.2. Ill-formed to use `this` in any way

Another possibility is to statically detect if the predicate in the precondition of a constructor or a postcondition in the destructor refers to `this`, even implicitly, and make such programs ill-formed. This includes things like `sizeof(*this)` or capturing `this` in a lambda. This has the potential to ban technically valid assertions that only need to read the address of the object but not its state.

Overall, however, this option seems a solution more in the spirit of the contracts design: do not introduce new reasons for undefined behavior. We already constrained what you can do in a predicate expression: names are implicitly `const` which prevents calling a lot of functions that you would be able to call in normal expressions.

While going with undefined behavior is a necessity for function-try-blocks (in the `try`-block we just have to allow any statement, and cannot arbitrarily filter them out), in the context of contract assertion, which are a separate feature with its own specific rules, we can afford to go the "ill-formed" way.

The contracts implementation in GCC 13 disallows the usage of `this` specifier in constructor preconditions and destructor postconditions. (See [this Compiler Explorer example](#).)

It should be noted that while most of the UB cases could be turned into ill-formed programs, some situations (e.g., when we obtain the pointer to our object by other means than `this`) cannot be statically enforced and will have to remain UB.

2.3. Ill-formed to call subobject non-static member functions

This is a less restrictive version of 2.2 above that allows operating on the address `this` but still prevents things that clearly read the object state: invoking non-static member functions on base and member subobjects.

The only problem with it is that it is not implementable. You cannot detect the invocation of a member function that is hidden behind another function call:

```
void f(struct X* x); // mystery

struct X
{
    X()
        pre(f(this))    // reads object state?
    ;
};
```

The specification for *function-try-block* can afford to only constrain referring to non-static member functions of subobjects, because it calls it undefined behavior, and for that no compiler checking is required.

3. Proposal

This paper does not state a preference between options 2.1 and 2.2. Instead we would like SG21 to chose which one is preferred.

4. Wording

We provide wording for both options 2.1 and 2.2. The proposed wording is relative to the wording proposed in [\[P2900R6\]](#).

4.1. Wording for the undefined behavior case

Modify [\[class.base.init\]/16](#) as follows.

Member functions (including virtual member functions, [\[class.virtual\]](#)) can be called for an object under construction or under destruction. Similarly, an object under construction can be the

operand of the `typeid` operator ([\[expr.typeid\]](#)) or of a `dynamic_cast` ([\[expr.dynamic.cast\]](#)). However, if these operations are performed

- in a *ctor-initializer* (or in a function called directly or indirectly from a *ctor-initializer*) before all the *mem-initializers* for base classes have completed, or
- in a precondition assertion of the constructor, or
- in a postcondition assertion of the destructor,

the program has undefined behavior.

Modify [\[class.ctor\]/1](#) as follows.

For an object with a non-trivial constructor, referring to any non-static member or base class of the object before the constructor begins execution results in undefined behavior. [Note: The evaluation of a constructor precondition assertion is considered part of constructor execution. — end note] For an object with a non-trivial destructor, referring to any non-static member or base class of the object after the destructor finishes execution results in undefined behavior. [Note: The evaluation of a destructor postcondition assertion is considered part of destructor execution. — end note]

Modify [\[class.ctor\]/4](#) as follows.

Member functions, including virtual functions ([\[class.virtual\]](#)), can be called during construction or destruction ([\[class.base.init\]](#)) and while evaluating a function contract assertion [dcl.contract.func]. When a virtual function is called directly or indirectly from a constructor or from a destructor, including during the construction or destruction of the class's non-static data members or during the evaluation of a postcondition assertion of a constructor or a precondition assertion of a destructor [dcl.contract.func], and the object to which the call applies is the object (call it *x*) under construction or destruction, the function called is the final overrider in the constructor's or destructor's class and not one overriding it in a more-derived class. If the virtual function call uses an explicit class member access ([\[expr.ref\]](#)) and the object expression refers to the complete object of *x* or one of that object's base class subobjects but not *x* or one of its base class subobjects, the behavior is undefined.

4.2. Wording for the ill-formed case

Apply all the changes from 4.1.

Modify [\[dcl.contract.func\]](#) as follows.

The predicate of a precondition assertion of a constructor shall not reference `this` or the `*this` object explicitly or implicitly. The predicate of a postcondition assertion of a destructor shall not reference `this` or the `*this` object explicitly or implicitly.

When a set of function contract assertions are *evaluated in sequence*, for any two function contract assertions *X* and *Y* in the set, the evaluation of *X* is sequenced before the evaluation of *Y* if the *function-contract-specifier* introducing *X* lexically precedes the one introducing *Y*.

5. Acknowledgments

Peter Brett observed this gap in the specification of contracts and suggested the wording changes for the case 2.1.

Joshua Berne reviewed the document and suggested wording changes.

Jens Maurer and Gašper Ažman reviewed the document, and contributed to its quality.

6. References

- [P2900R6] — Joshua Berne, Timur Doumler, Andrzej Krzemiński "Contracts for C++", (["https://isocpp.org/files/papers/P2900R6.pdf"](https://isocpp.org/files/papers/P2900R6.pdf)).